CSA4704 - DEEP LEARNING
Lab Experiments

Name: Harini. R
Register no: 192324108

Experiment 11: CNN for Image Classification
Problem Statement: A recycling plant wants a camera system that sorts waste on a conveyor belt into categories automatically. A garbage-classification image dataset (cardboard, glass, metal, paper, plastic, trash), or CIFAR-10 as a substitute. The objective is to build and train a CNN for multi-class image classification. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Load, resize and normalise the images; split into train, validation and test sets.
2. Build a CNN with convolution, pooling, flatten and dense layers.
3. Train with categorical cross-entropy and plot accuracy and loss curves.
4. Show the confusion matrix and sample misclassifications.
5. Visualise the feature maps of the first convolution layer.

Formulas:

A CNN learns visual features automatically from images.

- Convolution: Detects patterns such as edges, textures, and shapes.
- Max pooling: Reduces spatial dimensions by retaining strong activations.
- Flatten: Converts feature maps into a one-dimensional vector.
- Dense layer: Uses learned features to classify the image.
- Softmax: Produces probabilities across the 10 classes.
- Categorical cross-entropy: Measures classification error.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

tf.keras.utils.set_random_seed(42)

# 1. Load CIFAR-10
(x_all, y_all), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
```

```
]

# Split original training data into training and validation
x_train, x_val, y_train, y_val = train_test_split(
    x_all, y_all.ravel(),
    test_size=0.15,
    random_state=42,
    stratify=y_all.ravel()
)

y_test = y_test.ravel()

# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_val = x_val.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Training:", x_train.shape)
print("Validation:", x_val.shape)
print("Testing:", x_test.shape)

# Display sample images
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis("off")
plt.tight_layout()
plt.show()

# 2. Build CNN
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),

    tf.keras.layers.Conv2D(32, (3, 3), activation="relu",
                           padding="same", name="conv1"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
                           padding="same"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(128, (3, 3), activation="relu",
```

```
                    padding="same"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# 3. Train CNN
history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=15,
    batch_size=64,
    verbose=1
)

# Plot accuracy and loss
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history["accuracy"], label="Training")
axes[0].plot(history.history["val_accuracy"], label="Validation")
axes[0].set_title("Accuracy vs Epochs")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history["loss"], label="Training")
axes[1].plot(history.history["val_loss"], label="Validation")
axes[1].set_title("Loss vs Epochs")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True)
```

```
plt.tight_layout()
plt.show()

# 4. Test evaluation
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0
)
print("Test loss:", round(test_loss, 4))
print("Test accuracy:", round(test_accuracy, 4))

# Confusion matrix
probabilities = model.predict(x_test, verbose=0)
predictions = np.argmax(probabilities, axis=1)

cm = confusion_matrix(y_test, predictions)
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)
display.plot(xticks_rotation=45, values_format="d")
plt.title("CIFAR-10 Confusion Matrix")
plt.tight_layout()
plt.show()

# Display misclassified examples
wrong = np.where(predictions != y_test)[0]
num_show = min(10, len(wrong))

plt.figure(figsize=(12, 5))
for i, idx in enumerate(wrong[:num_show]):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[idx])
    plt.title(
        f"True: {class_names[y_test[idx]]}\n"
        f"Pred: {class_names[predictions[idx]]}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()

# 5. Visualize feature maps of the first convolution layer
feature_model = tf.keras.Model(
```

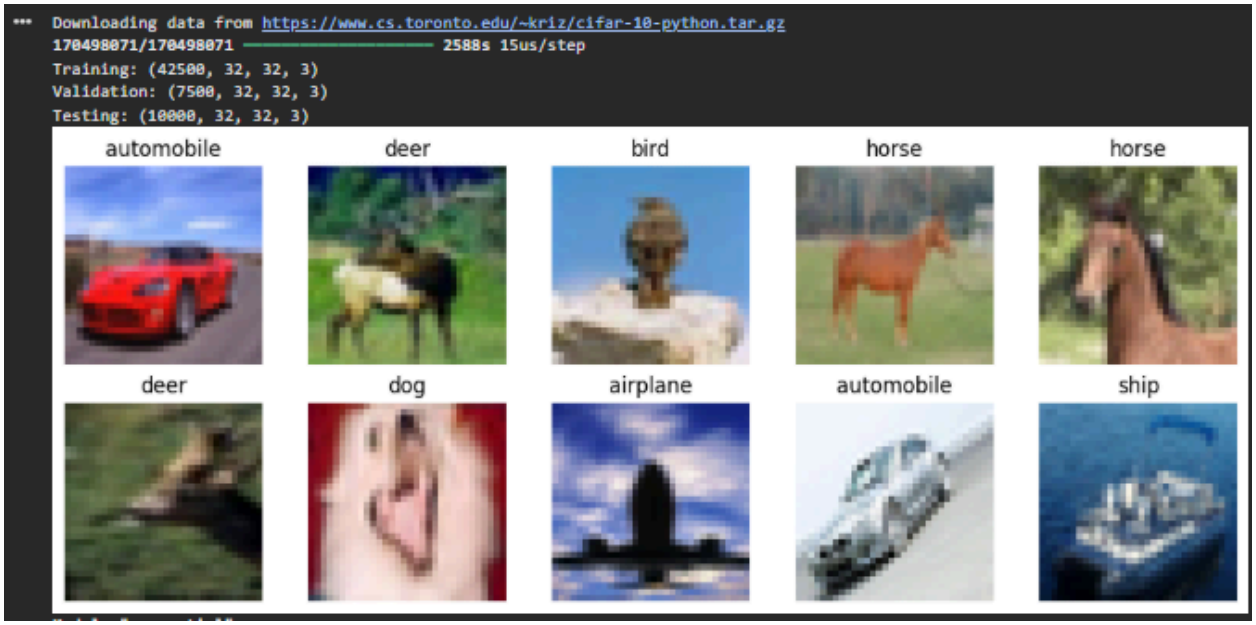
```
    inputs=model.inputs,
    outputs=model.get_layer("conv1").output
)

feature_maps = feature_model.predict(x_test[:1], verbose=0)
num_maps = min(16, feature_maps.shape[-1])

plt.figure(figsize=(10, 10))
for i in range(num_maps):
    plt.subplot(4, 4, i + 1)
    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")
    plt.title(f"Filter {i + 1}")
    plt.axis("off")

plt.suptitle("First Convolution Layer Feature Maps")
plt.tight_layout()
plt.show()
```

Output:

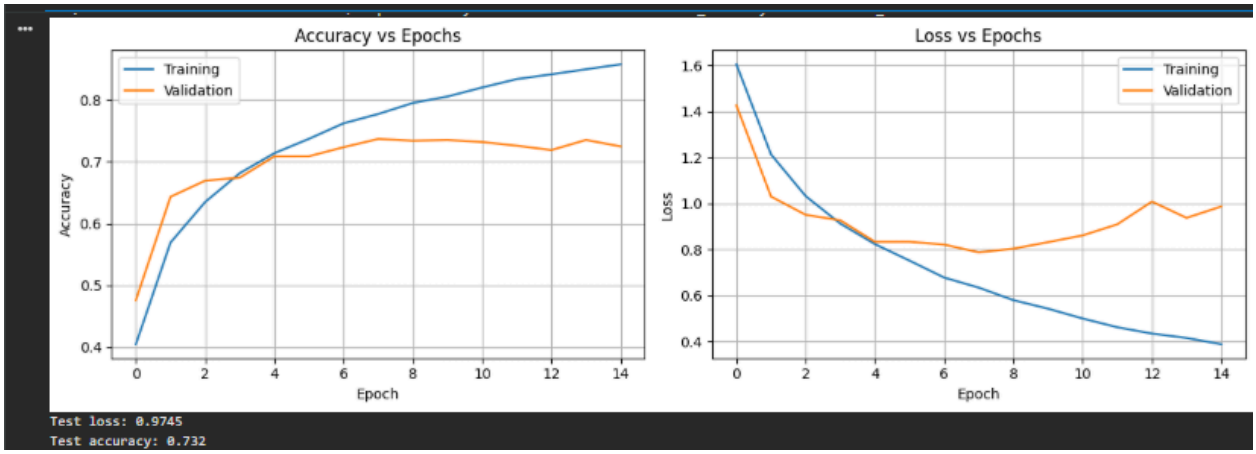


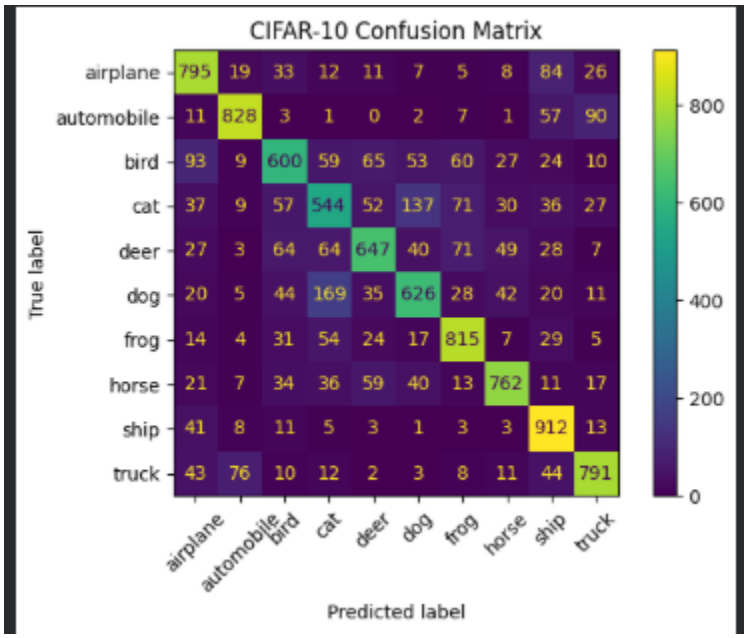
Model: "sequential"

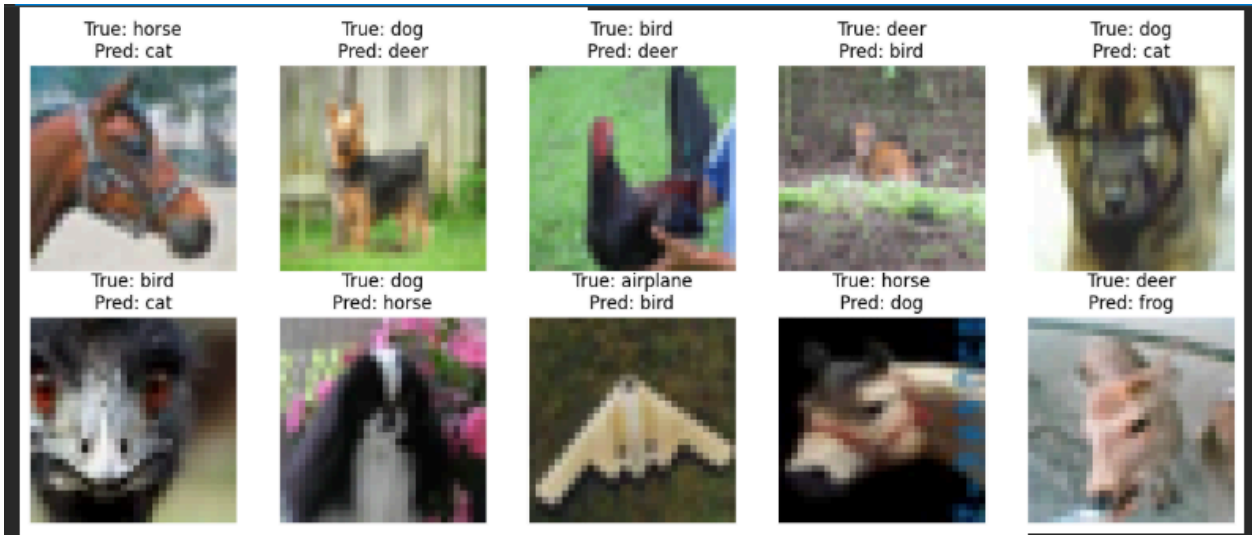
Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_1 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

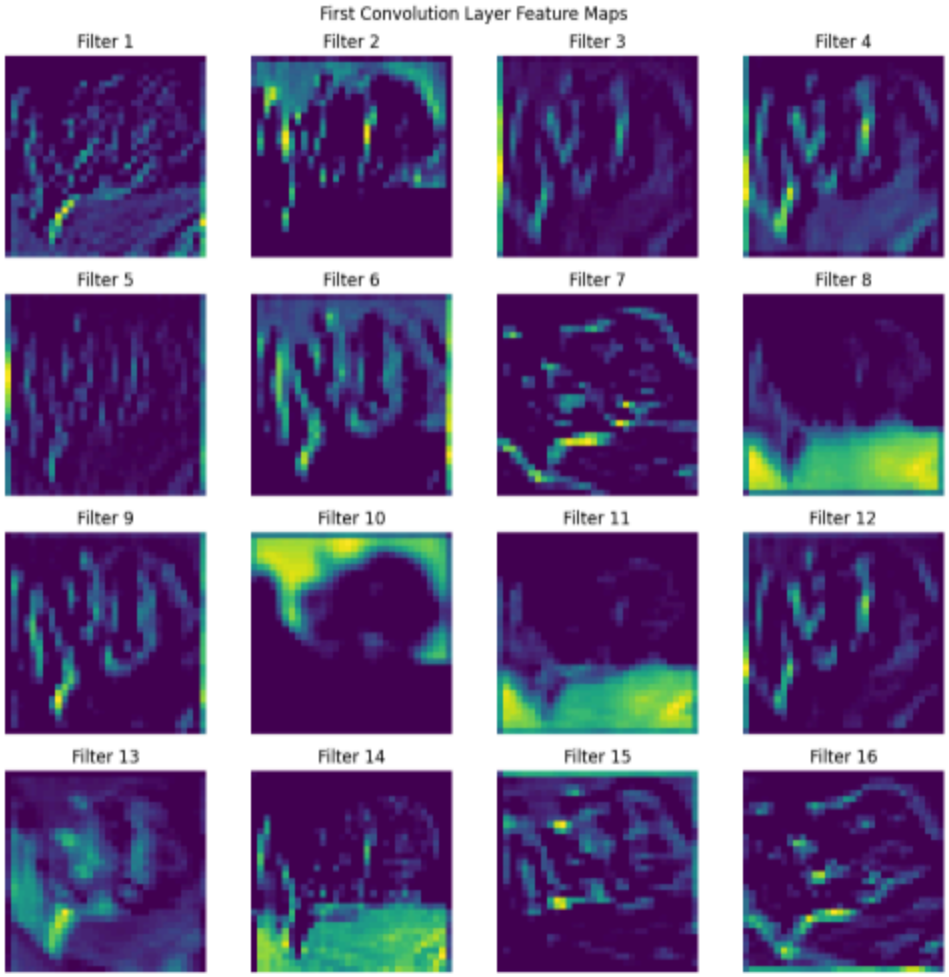
Total params: 356,810 (1.36 MB)
Trainable params: 356,810 (1.36 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 106s 156ms/step - accuracy: 0.4049 - loss: 1.6058 - val_accuracy: 0.4761 - val_loss: 1.4274
Epoch 2/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 99s 148ms/step - accuracy: 0.5692 - loss: 1.2141 - val_accuracy: 0.6432 - val_loss: 1.0311
Epoch 3/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 147s 155ms/step - accuracy: 0.6347 - loss: 1.0329 - val_accuracy: 0.6692 - val_loss: 0.9515
Epoch 4/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 98s 147ms/step - accuracy: 0.6814 - loss: 0.9135 - val_accuracy: 0.6741 - val_loss: 0.9278
Epoch 5/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 98s 147ms/step - accuracy: 0.7136 - loss: 0.8243 - val_accuracy: 0.7085 - val_loss: 0.8342
Epoch 6/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 141s 146ms/step - accuracy: 0.7369 - loss: 0.7524 - val_accuracy: 0.7087 - val_loss: 0.8343
Epoch 7/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 143s 148ms/step - accuracy: 0.7620 - loss: 0.6788 - val_accuracy: 0.7233 - val_loss: 0.8222
Epoch 8/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 142s 149ms/step - accuracy: 0.7770 - loss: 0.6350 - val_accuracy: 0.7367 - val_loss: 0.7884
Epoch 9/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 147s 156ms/step - accuracy: 0.7948 - loss: 0.5806 - val_accuracy: 0.7337 - val_loss: 0.8043
Epoch 10/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 104s 157ms/step - accuracy: 0.8052 - loss: 0.5435 - val_accuracy: 0.7349 - val_loss: 0.8327
Epoch 11/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 99s 149ms/step - accuracy: 0.8198 - loss: 0.5008 - val_accuracy: 0.7317 - val_loss: 0.8621
Epoch 12/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 97s 147ms/step - accuracy: 0.8334 - loss: 0.4628 - val_accuracy: 0.7257 - val_loss: 0.9103
Epoch 13/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 142s 146ms/step - accuracy: 0.8411 - loss: 0.4353 - val_accuracy: 0.7187 - val_loss: 1.0091
Epoch 14/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 143s 148ms/step - accuracy: 0.8493 - loss: 0.4163 - val_accuracy: 0.7351 - val_loss: 0.9383
Epoch 15/15
665/665 ━━━━━━━━━━━━━━━━━━━━ 97s 146ms/step - accuracy: 0.8572 - loss: 0.3892 - val_accuracy: 0.7245 - val_loss: 0.9878
```


Accuracy vs Epochs
Training
Validation
Accuracy
Epoch
Loss vs Epochs
Training
Validation
Loss
Epoch
Test loss: 0.9745
Test accuracy: 0.732


CIFAR-10 Confusion Matrix
airplane 795 19 33 12 11 7 5 8 84 26
automobile 11 828 3 1 0 2 7 1 57 90
bird 93 9 600 59 65 53 60 27 24 10
cat 37 9 57 544 52 137 71 30 36 27
deer 27 3 64 64 647 40 71 49 28 7
dog 20 5 44 169 35 626 28 42 20 11
frog 14 4 31 54 24 17 815 7 29 5
horse 21 7 34 36 59 40 13 762 11 17
ship 41 8 11 5 3 1 3 3 912 13
truck 43 76 10 12 2 3 8 11 44 791
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
True label
Predicted label
800
600
400
200
0


True: horse
Pred: cat
True: dog
Pred: deer
True: bird
Pred: deer
True: deer
Pred: bird
True: dog
Pred: cat
True: bird
Pred: cat
True: dog
Pred: horse
True: airplane
Pred: bird
True: horse
Pred: dog
True: deer
Pred: frog



Experiment 12: CNN Architecture and Hyperparameter Comparison
Problem Statement: The recycling plant's CNN must run on a low-cost edge device. The engineer must balance accuracy against model size and training time. The same waste or CIFAR-10 dataset from Experiment 11. The objective is to compare CNN architectures and hyperparameters systematically. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Vary the number of convolution layers (2, 3, 4).
2. Vary filter size (3×3, 5×5) and filter count.
3. Compare max pooling and average pooling, with and without batch normalisation.
4. Vary learning rate and batch size.
5. Tabulate accuracy, parameter count and training time for each configuration.

Formulas:

- More convolution layers can learn increasingly complex features.
- Larger filters increase the receptive field but usually add parameters.
- Batch normalization can stabilize training.
- Max pooling selects strong activations; average pooling averages local activations.

- Learning rate controls update size, while batch size controls how many examples contribute to each update.

Code:

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

tf.keras.utils.set_random_seed(42)

# Load and normalize CIFAR-10
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = y_train.ravel()
y_test = y_test.ravel()

# Use subsets to reduce runtime
x_sub, y_sub = x_train[:10000], y_train[:10000]
x_val, y_val = x_train[10000:12000], y_train[10000:12000]

def build_cnn(depth=2, filters=32, kernel=3,
              pooling="max", batch_norm=False, lr=0.001):
    layers = [tf.keras.layers.Input(shape=(32, 32, 3))]

    for i in range(depth):
        count = filters * (2 ** min(i, 2))
        layers.append(tf.keras.layers.Conv2D(
            count, kernel, padding="same", use_bias=not batch_norm
        ))

        if batch_norm:
            layers.append(tf.keras.layers.BatchNormalization())

        layers.append(tf.keras.layers.Activation("relu"))

        if pooling == "max":
            layers.append(tf.keras.layers.MaxPooling2D(2))
        else:
            layers.append(tf.keras.layers.AveragePooling2D(2))
```

```
    layers.extend([
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation="softmax")
    ])

    model = tf.keras.Sequential(layers)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

# Each configuration changes one or more settings.
# Together, these runs demonstrate the requested comparisons.
configs = [
    {"name": "2 layers, 3x3, max", "depth": 2, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 64},
    {"name": "3 layers, 3x3, max", "depth": 3, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 64},
    {"name": "4 layers, 3x3, max", "depth": 4, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 64},
    {"name": "3 layers, 5x5", "depth": 3, "filters": 32,
     "kernel": 5, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 64},
    {"name": "3 layers, average pool", "depth": 3, "filters": 32,
     "kernel": 3, "pooling": "average", "batch_norm": False,
     "lr": 0.001, "batch_size": 64},
    {"name": "3 layers, batch norm", "depth": 3, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": True,
     "lr": 0.001, "batch_size": 64},
    {"name": "3 layers, LR=0.01", "depth": 3, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.01, "batch_size": 64},
    {"name": "3 layers, batch=128", "depth": 3, "filters": 32,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 128},
    {"name": "3 layers, 64 filters", "depth": 3, "filters": 64,
     "kernel": 3, "pooling": "max", "batch_norm": False,
     "lr": 0.001, "batch_size": 64}
]
```

```
results = []

for cfg in configs:
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(42)

    model = build_cnn(
        depth=cfg["depth"],
        filters=cfg["filters"],
        kernel=cfg["kernel"],
        pooling=cfg["pooling"],
        batch_norm=cfg["batch_norm"],
        lr=cfg["lr"]
    )

    start = time.perf_counter()

    model.fit(
        x_sub, y_sub,
        validation_data=(x_val, y_val),
        epochs=3,
        batch_size=cfg["batch_size"],
        verbose=0
    )

    elapsed = time.perf_counter() - start
    val_loss, val_acc = model.evaluate(x_val, y_val, verbose=0)

    results.append({
        "Configuration": cfg["name"],
        "Validation accuracy": val_acc,
        "Validation loss": val_loss,
        "Parameters": model.count_params(),
        "Training time (seconds)": elapsed
    })

results_df = pd.DataFrame(results)
print(results_df.round(4).to_string(index=False))

# Plot accuracy against model size
plt.figure(figsize=(9, 5))
plt.scatter(
    results_df["Parameters"],
```

```
    results_df["Validation accuracy"]
)

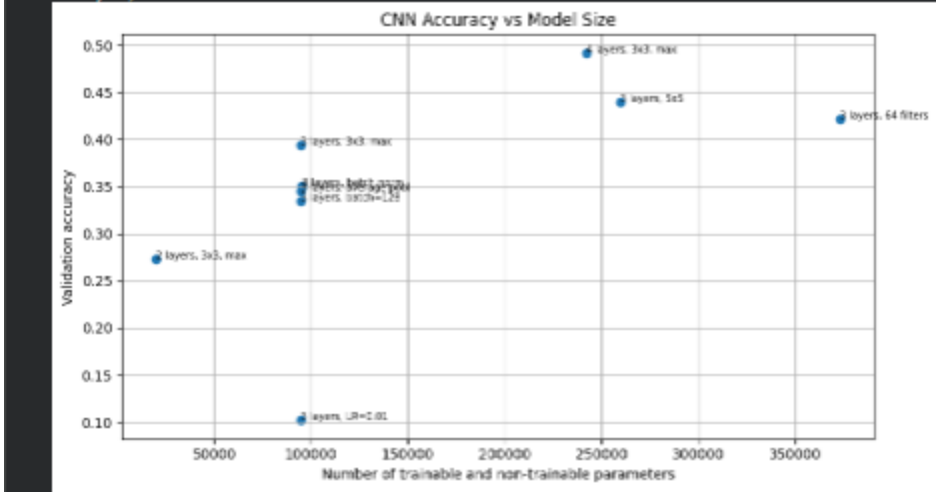
for _, row in results_df.iterrows():
    plt.annotate(
        row["Configuration"],
        (row["Parameters"], row["Validation accuracy"]),
        fontsize=7
    )

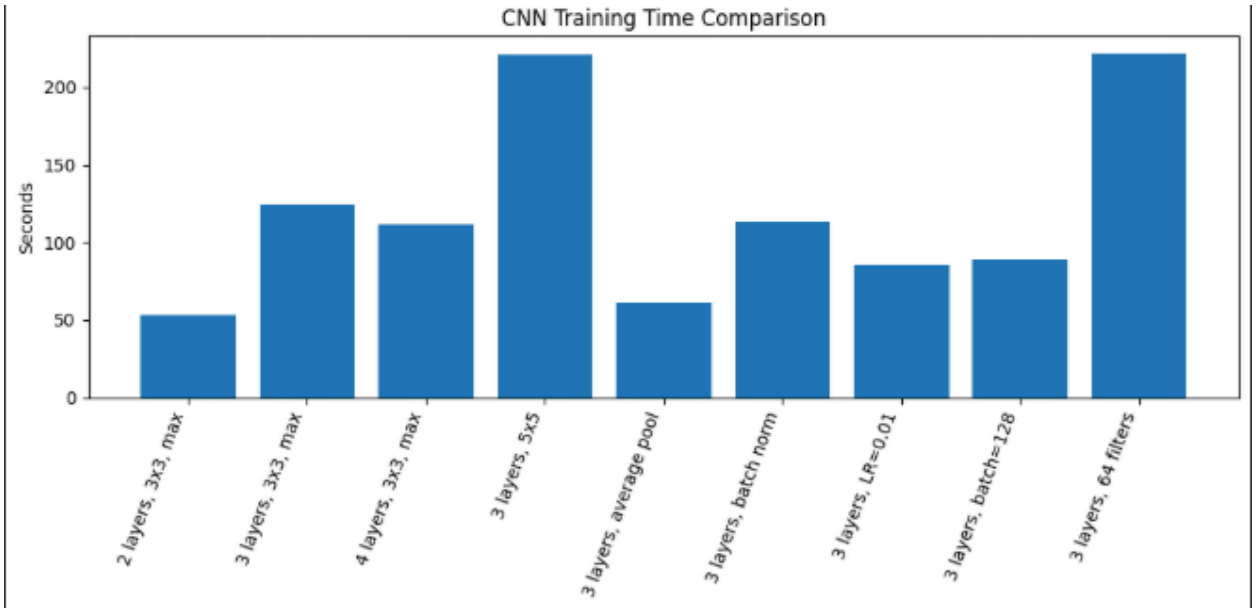
plt.xlabel("Number of trainable and non-trainable parameters")
plt.ylabel("Validation accuracy")
plt.title("CNN Accuracy vs Model Size")
plt.grid(True)
plt.tight_layout()
plt.show()

# Plot training time
plt.figure(figsize=(10, 5))
plt.bar(
    results_df["Configuration"],
    results_df["Training time (seconds)"]
)
plt.xticks(rotation=70, ha="right")
plt.ylabel("Seconds")
plt.title("CNN Training Time Comparison")
plt.tight_layout()
plt.show()
```

Output:

	Configuration	Validation accuracy	Validation loss	Parameters	Training time (seconds)
2 layers, 3x3, max		0.2735	1.8954	28042	53.3435
3 layers, 3x3, max		0.3945	1.6737	94538	124.3839
4 layers, 3x3, max		0.4920	1.4013	242122	111.7332
3 layers, 5x5		0.4400	1.5449	259914	220.8651
3 layers, average pool		0.3458	1.7410	94538	61.2843
3 layers, batch norm		0.3505	1.8321	95210	113.0262
3 layers, LR=0.01		0.1020	2.3039	94538	85.8984
3 layers, batch=128		0.3345	1.7521	94538	88.6368
3 layers, 64 filters		0.4215	1.6052	373386	221.8846





CO3 – Sequence Models (RNN, LSTM, GRU)

Experiment 13: Sequential Dataset Preparation and RNN Prediction

Problem Statement: A state electricity board must forecast next-hour power demand to schedule generation and avoid shortages. Hourly electricity consumption data for one year (or a daily temperature series as a substitute). The objective is to convert a time series into supervised sequences and predict the next value with a SimpleRNN. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Load the series with Pandas; handle missing values and scale it.

2. Create sliding windows (for example, 24 past hours → next hour).
3. Split chronologically into train and test sets.
4. Build and train a SimpleRNN model in Keras.
5. Plot actual against predicted demand and report MAE and RMSE.

Formulas:

A sliding window uses a fixed number of past observations to predict the next observation.

For example, with a window size of 24:

$$[x_{t-23}, \ldots, x_t] \rightarrow x_{t+1}$$

The SimpleRNN maintains a hidden state representing information from earlier time steps. The model learns to predict the next demand value based on past observations.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

tf.keras.utils.set_random_seed(42)
np.random.seed(42)

# 1. Generate one year of hourly synthetic electricity demand
hours = 365 * 24
timestamps = pd.date_range(
    start="2025-01-01", periods=hours, freq="h"
)

t = np.arange(hours)
daily_cycle = 15 * np.sin(2 * np.pi * t / 24)
weekly_cycle = 5 * np.sin(2 * np.pi * t / (24 * 7))
yearly_trend = 0.015 * t

demand = (
    100 + daily_cycle + weekly_cycle + yearly_trend
    + np.random.normal(0, 3, hours)
)
```

```
df = pd.DataFrame({
    "timestamp": timestamps,
    "demand": demand
}).set_index("timestamp")

# Handle missing values
df["demand"] = df["demand"].interpolate().ffill().bfill()

plt.figure(figsize=(12, 4))
plt.plot(df.index[:7*24], df["demand"].iloc[:7*24])
plt.title("Example Week of Electricity Demand")
plt.xlabel("Time")
plt.ylabel("Demand (synthetic units)")
plt.grid(True)
plt.show()

# 2. Chronological train/test split
values = df["demand"].to_numpy().reshape(-1, 1)
split = int(len(values) * 0.8)

# Fit scaler only on training data
scaler = MinMaxScaler()
scaler.fit(values[:split])
scaled = scaler.transform(values)

def create_sequences(data, window):
    X, y = [], []
    for i in range(window, len(data)):
        X.append(data[i-window:i])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

window = 24
X_all, y_all = create_sequences(scaled, window)

# Target timestamps determine split; no random shuffling.
target_indices = np.arange(window, len(values))
train_mask = target_indices < split
test_mask = target_indices >= split

X_train, y_train = X_all[train_mask], y_all[train_mask]
X_test, y_test = X_all[test_mask], y_all[test_mask]
```

```
print("Training sequences:", X_train.shape)
print("Testing sequences:", X_test.shape)

# 3. Build SimpleRNN
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(window, 1)),
    tf.keras.layers.SimpleRNN(32, activation="tanh"),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=20,
    batch_size=32,
    shuffle=False,
    verbose=1
)

# Plot training and validation loss
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="Training")
plt.plot(history.history["val_loss"], label="Validation")
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.title("SimpleRNN Training Loss")
plt.legend()
plt.grid(True)
plt.show()

# 4. Predict and reverse scaling
pred_scaled = model.predict(X_test, verbose=0)

pred = scaler.inverse_transform(pred_scaled).ravel()
actual = scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()

# 5. Evaluate
mae = mean_absolute_error(actual, pred)
rmse = np.sqrt(mean_squared_error(actual, pred))

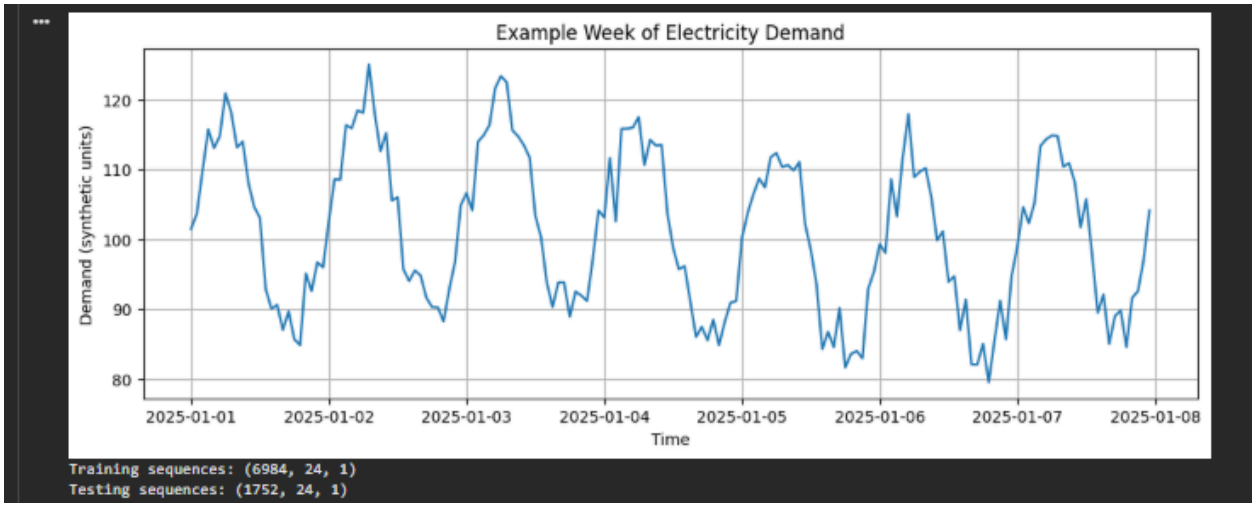
print("Test MAE:", round(mae, 3))
```

```
print("Test RMSE:", round(rmse, 3))

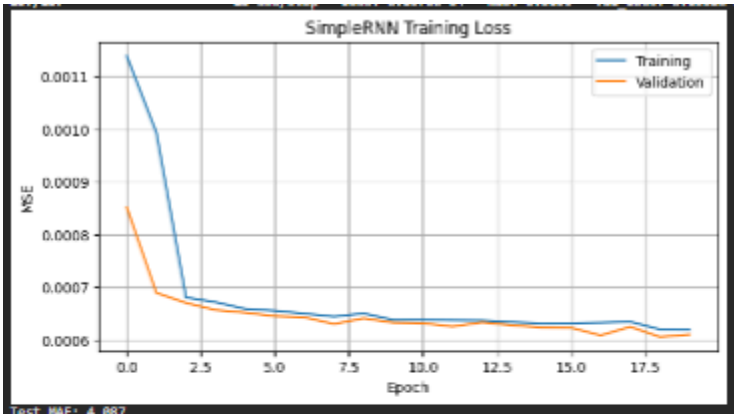
# Plot actual vs predicted
test_times = df.index[target_indices[test_mask]]

plt.figure(figsize=(12, 5))
plt.plot(test_times[:300], actual[:300], label="Actual")
plt.plot(test_times[:300], pred[:300], label="Predicted")
plt.xlabel("Time")
plt.ylabel("Demand")
plt.title("Actual vs Predicted Electricity Demand")
plt.legend()
plt.grid(True)
plt.show()
```

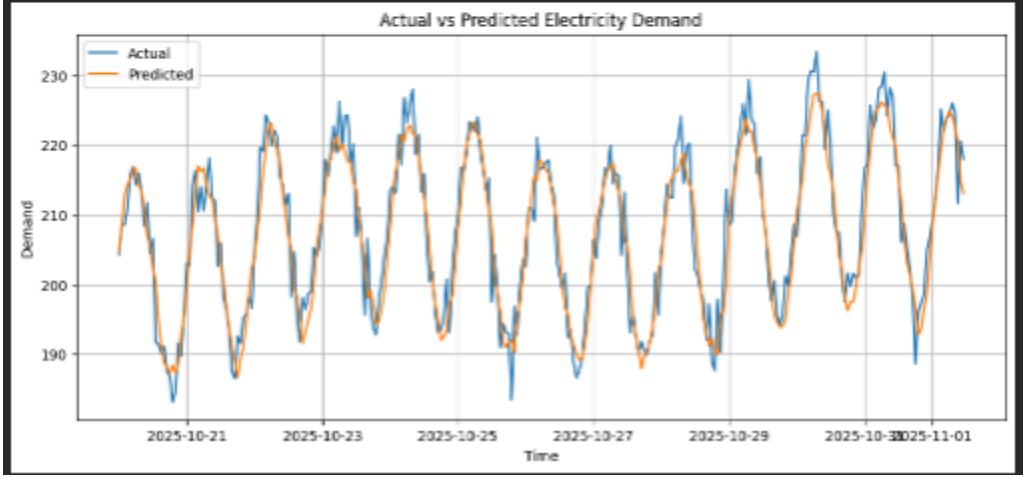
Output:



```
Epoch 1/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 7s 13ms/step - loss: 0.0011 - mae: 0.0265 - val_loss: 8.5228e-04 - val_mae: 0.0234
Epoch 2/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - loss: 9.9359e-04 - mae: 0.0246 - val_loss: 6.8947e-04 - val_mae: 0.0209
Epoch 3/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 6.8072e-04 - mae: 0.0209 - val_loss: 6.7035e-04 - val_mae: 0.0206
Epoch 4/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 6.7172e-04 - mae: 0.0207 - val_loss: 6.5676e-04 - val_mae: 0.0205
Epoch 5/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 6.5894e-04 - mae: 0.0205 - val_loss: 6.5175e-04 - val_mae: 0.0204
Epoch 6/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.5548e-04 - mae: 0.0204 - val_loss: 6.4510e-04 - val_mae: 0.0203
Epoch 7/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.5008e-04 - mae: 0.0203 - val_loss: 6.4295e-04 - val_mae: 0.0203
Epoch 8/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.4429e-04 - mae: 0.0203 - val_loss: 6.3020e-04 - val_mae: 0.0201
Epoch 9/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.5034e-04 - mae: 0.0204 - val_loss: 6.4098e-04 - val_mae: 0.0203
Epoch 10/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3802e-04 - mae: 0.0202 - val_loss: 6.3274e-04 - val_mae: 0.0201
Epoch 11/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3821e-04 - mae: 0.0202 - val_loss: 6.3177e-04 - val_mae: 0.0201
Epoch 12/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3744e-04 - mae: 0.0202 - val_loss: 6.2546e-04 - val_mae: 0.0200
Epoch 13/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3694e-04 - mae: 0.0202 - val_loss: 6.3337e-04 - val_mae: 0.0201
Epoch 14/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 6.3391e-04 - mae: 0.0201 - val_loss: 6.2788e-04 - val_mae: 0.0201
Epoch 15/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 6.3131e-04 - mae: 0.0201 - val_loss: 6.2373e-04 - val_mae: 0.0200
Epoch 16/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3163e-04 - mae: 0.0201 - val_loss: 6.2318e-04 - val_mae: 0.0200
Epoch 17/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3276e-04 - mae: 0.0202 - val_loss: 6.0861e-04 - val_mae: 0.0197
Epoch 18/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.3460e-04 - mae: 0.0201 - val_loss: 6.2495e-04 - val_mae: 0.0200
Epoch 19/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.2000e-04 - mae: 0.0199 - val_loss: 6.0571e-04 - val_mae: 0.0197
Epoch 20/20
197/197 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.1970e-04 - mae: 0.0199 - val_loss: 6.1002e-04 - val_mae: 0.0198
```




```
Test MAE: 4.887
Test RMSE: 5.101
```



Experiment 14: Unfolded RNN Architecture Visualization
Problem Statement: A new ML engineer at a speech-analytics firm finds the recurrent loop hard to follow. The team lead asks for a diagram showing the RNN unrolled over time. A 5-word input sentence passed through a SimpleRNN with a 3-unit hidden state. The objective is to visualise an RNN unfolded across time steps and trace the hidden-state values. Use TensorFlow, Keras, NetworkX, Matplotlib.
Tasks:
1. Build a SimpleRNN in Keras and extract its input, recurrent and output weights.
2. Compute hidden states for each time step manually and confirm them against Keras.
3. Draw the unfolded network as a graph with NetworkX (nodes x1...x5, h1...h5, y1...y5).
4. Label edges with shared weights W_xh, W_hh and W_hy.
5. Plot the hidden-state values across time steps.

Formulas:

At each time step, a recurrent neural network computes

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

The output can be calculated as

$$y_t = W_{hy} h_t + b_y$$

The same input weights W_{xh} and recurrent weights W_{hh} are shared across all time steps. Unfolding shows the repeated computation as a sequence of connected nodes.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import networkx as nx

tf.keras.utils.set_random_seed(42)
np.set_printoptions(precision=4, suppress=True)

# Five-word sentence represented by illustrative scalar embeddings
words = ["The", "delivery", "arrived", "very", "late"]
x = np.array([0.2, 0.8, 0.5, 0.3, 0.9], dtype=np.float32)
X = x.reshape(1, 5, 1)

# 1. Build SimpleRNN with 3 hidden units
```

```
inputs = tf.keras.Input(shape=(5, 1))
rnn_layer = tf.keras.layers.SimpleRNN(
    3, activation="tanh", return_sequences=True,
    return_state=True, name="simple_rnn"
)

sequence_outputs, final_state = rnn_layer(inputs)
output_layer = tf.keras.layers.Dense(1, name="output")
outputs = output_layer(sequence_outputs)

model = tf.keras.Model(inputs, [outputs, sequence_outputs, final_state])

# Extract weights
W_xh, W_hh, b_h = rnn_layer.get_weights()
W_hy, b_y = output_layer.get_weights()

print("Input weights W_xh:\n", W_xh)
print("Recurrent weights W_hh:\n", W_hh)
print("Hidden bias:\n", b_h)
print("Output weights W_hy:\n", W_hy)
print("Output bias:\n", b_y)

# 2. Calculate hidden states manually
h = np.zeros(3)
manual_states = []
manual_outputs = []

for t in range(5):
    h = np.tanh(x[t] * W_xh[0] + h @ W_hh + b_h)
    manual_states.append(h.copy())

    y_t = h @ W_hy + b_y
    manual_outputs.append(y_t.copy())

manual_states = np.array(manual_states)
manual_outputs = np.array(manual_outputs)

keras_outputs, keras_states, keras_final = model(X, training=False)

keras_states = keras_states.numpy()[0]
keras_outputs = keras_outputs.numpy()[0]

print("\nManual hidden states:\n", manual_states)
print("\nKeras hidden states:\n", keras_states)
```

```
print("\nHidden states match:",
      np.allclose(manual_states, keras_states, atol=1e-5))

print("\nManual outputs:\n", manual_outputs.ravel())
print("Keras outputs:\n", keras_outputs.ravel())
print("Outputs match:",
      np.allclose(manual_outputs, keras_outputs, atol=1e-5))

# 3. Draw unfolded RNN graph
G = nx.DiGraph()
positions = {}

for t in range(5):
    x_node = f"x{t+1}"
    h_node = f"h{t+1}"
    y_node = f"y{t+1}"

    G.add_node(x_node, kind="input")
    G.add_node(h_node, kind="hidden")
    G.add_node(y_node, kind="output")

    positions[x_node] = (t, 2)
    positions[h_node] = (t, 1)
    positions[y_node] = (t, 0)

    G.add_edge(x_node, h_node, label="W_xh")
    G.add_edge(h_node, y_node, label="W_hy")

    if t > 0:
        G.add_edge(f"h{t}", h_node, label="W_hh")

plt.figure(figsize=(13, 5))

node_colors = []
for node in G.nodes():
    kind = G.nodes[node]["kind"]
    node_colors.append({
        "input": "lightblue",
        "hidden": "lightgreen",
        "output": "lightyellow"
    }[kind])

nx.draw_networkx(
    G, positions,
```

```
    node_color=node_colors,
    node_size=1500,
    font_size=10,
    arrows=True,
    arrowsize=18
)

edge_labels = nx.get_edge_attributes(G, "label")
nx.draw_networkx_edge_labels(
    G, positions,
    edge_labels=edge_labels,
    font_size=8
)

plt.title("SimpleRNN Unfolded Across Five Time Steps")
plt.axis("off")
plt.tight_layout()
plt.show()

# 4. Plot each hidden unit across time
plt.figure(figsize=(8, 4))

for unit in range(3):
    plt.plot(
        range(1, 6),
        manual_states[:, unit],
        marker="o",
        label=f"Hidden unit {unit+1}"
    )

plt.xticks(range(1, 6), words)
plt.xlabel("Input word / time step")
plt.ylabel("Hidden-state activation")
plt.title("Hidden States Across Time")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

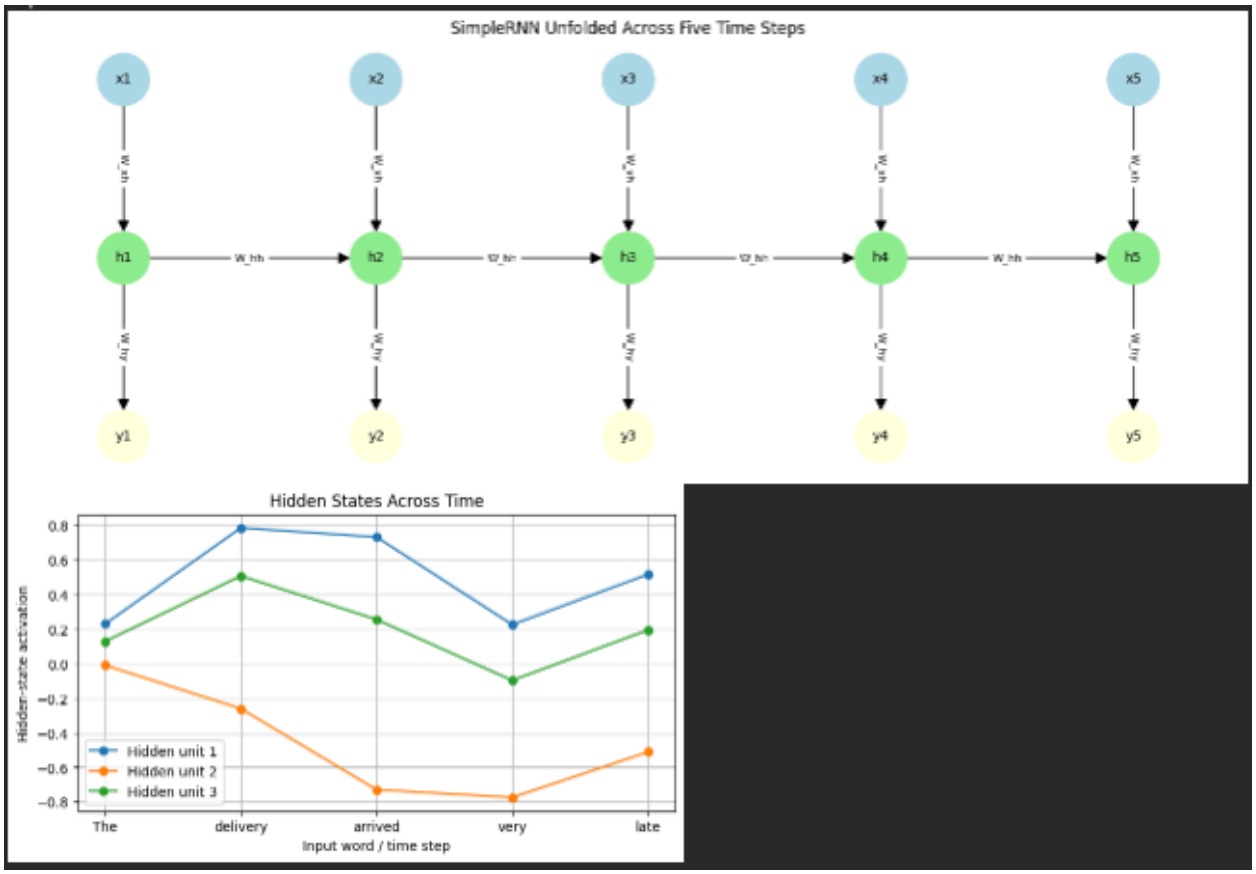
```
Input weights W_xh:
 [[ 1.1737 -0.044   0.6518]]
Recurrent weights W_hh:
 [[ 0.0854 -0.8704  0.4849]
 [ 0.5458  0.4481  0.7081]
 [ 0.8336 -0.2042 -0.5133]]
Hidden bias:
 [0. 0. 0.]
Output weights W_hy:
 [[0.7739]
 [0.3342]
 [0.0029]]
Output bias:
 [0.]

Manual hidden states:
 [[ 0.2385 -0.0088  0.1296]
 [ 0.7864 -0.2602  0.5083]
 [ 0.7333 -0.7291  0.2563]
 [ 0.2264 -0.7741 -0.0964]
 [ 0.5174 -0.5109  0.1953]]

Keras hidden states:
 [[ 0.2385 -0.0088  0.1296]
 [ 0.7864 -0.2602  0.5083]
 [ 0.7333 -0.7291  0.2563]
 [ 0.2264 -0.7741 -0.0964]
 [ 0.5174 -0.5109  0.1953]]

Hidden states match: True

Manual outputs:
 [ 0.1758  0.5231  0.3245 -0.0837  0.2303]
Keras outputs:
 [ 0.1758  0.5231  0.3245 -0.0837  0.2303]
```



Experiment 15: BPTT Flow and Gradient Analysis

Problem Statement: A stock-analytics firm finds its RNN ignores price movements from more than a few days ago. The team suspects the gradients are vanishing. Daily closing prices of a stock index over 3 years, with sequence lengths of 10, 50 and 100 days. The objective is to trace

backpropagation through time and measure gradient magnitude at each time step. Use TensorFlow, Keras, NumPy, Matplotlib.
Tasks:
1. Build a SimpleRNN and use tf.GradientTape to compute gradients of the loss.
2. Record the gradient norm with respect to the hidden state at each time step.
3. Plot gradient norm against time step for each sequence length.
4. Compare tanh and ReLU activations.
5. Explain vanishing and exploding gradients from the plots.

Formulas:

Backpropagation through time (BPTT) computes gradients by propagating prediction errors backward through the unrolled recurrent network.

- Vanishing gradients: Gradient magnitudes become extremely small as they travel backward, making long-term dependencies difficult to learn.
- Exploding gradients: Gradient magnitudes grow excessively, potentially destabilizing training.
- Tanh: Outputs values between -1 and 1; repeated multiplication by small derivatives can cause vanishing gradients.
- ReLU: Can preserve gradients for positive activations but can also lead to unstable growth in recurrent networks.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

np.random.seed(42)
tf.keras.utils.set_random_seed(42)

# 1. Generate 3 years of synthetic daily closing prices
n = 3 * 365
returns = np.random.normal(0.0003, 0.015, n)
prices = 100 * np.exp(np.cumsum(returns))

df = pd.DataFrame({
   "date": pd.date_range("2023-01-01", periods=n, freq="D"),
   "close": prices
})

plt.figure(figsize=(10, 4))
plt.plot(df["date"], df["close"])
plt.title("Synthetic Daily Stock-Index Closing Prices")
```

```
plt.xlabel("Date")
plt.ylabel("Closing price")
plt.grid(True)
plt.show()

# 2. Build a SimpleRNN to extract recurrent weights
# The actual gradient experiment below explicitly unrolls the recurrence.
def make_rnn_weights(activation, units=16):
    layer = tf.keras.layers.SimpleRNN(
        units, activation=activation, return_sequences=True
    )
    layer.build((None, None, 1))
    kernel, recurrent_kernel, bias = layer.get_weights()
    return kernel, recurrent_kernel, bias

# Prepare normalized price changes
returns_scaled = (returns - returns.mean()) / (returns.std() + 1e-8)
series = returns_scaled.astype(np.float32)

# 3. Explicitly unroll an RNN so intermediate hidden-state
# gradients can be measured with GradientTape.
def analyze_gradients(sequence_length, activation_name,
                      seed=42, units=16):
    tf.keras.utils.set_random_seed(seed)

    kernel, recurrent_kernel, bias = make_rnn_weights(
        activation_name, units
    )

    # Convert extracted Keras RNN weights to tensors
    W_xh = tf.constant(kernel, dtype=tf.float32)
    W_hh = tf.constant(recurrent_kernel, dtype=tf.float32)
    b_h = tf.constant(bias, dtype=tf.float32)

    # One fixed input sequence
    seq = tf.constant(
        series[-sequence_length:].reshape(sequence_length, 1),
        dtype=tf.float32
    )

    with tf.GradientTape() as tape:
        h = tf.zeros((1, units), dtype=tf.float32)
        hidden_states = []
```

```
        for t in range(sequence_length):
            z = seq[t:t+1] @ W_xh + h @ W_hh + b_h

            if activation_name == "tanh":
                h = tf.math.tanh(z)
            elif activation_name == "relu":
                h = tf.nn.relu(z)
            else:
                raise ValueError("Unsupported activation")

            hidden_states.append(h)

        # Keep the full list of states connected in the graph.
        # Prediction loss depends on the final hidden state.
        target = tf.constant([[0.0]], dtype=tf.float32)
        output_weights = tf.ones((units, 1), dtype=tf.float32) / units
        prediction = h @ output_weights
        loss = tf.reduce_mean(tf.square(prediction - target))

    # Gradient of final loss with respect to each hidden state
    grads = tape.gradient(loss, hidden_states)

    norms = np.array([
        float(tf.norm(g).numpy()) if g is not None else 0.0
        for g in grads
    ])

    return norms, float(loss.numpy())

# 4. Compare sequence lengths and activations
sequence_lengths = [10, 50, 100]
activations = ["tanh", "relu"]
gradient_results = {}

for activation in activations:
    for length in sequence_lengths:
        norms, loss = analyze_gradients(length, activation)
        gradient_results[(activation, length)] = norms

        print(
            f"Activation={activation}, length={length}, "
            f"initial-state gradient norm={norms[0]:.6e}, "
            f"final-state gradient norm={norms[-1]:.6e}"
        )
```

```
# 5. Plot gradient magnitude against time step
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, activation in zip(axes, activations):
    for length in sequence_lengths:
        norms = gradient_results[(activation, length)]

        # Avoid zero values on a logarithmic axis
        ax.plot(
            np.arange(1, length + 1),
            np.maximum(norms, 1e-12),
            label=f"Length {length}"
        )

    ax.set_yscale("log")
    ax.set_xlabel("Time step (earlier to later)")
    ax.set_ylabel("Gradient norm (log scale)")
    ax.set_title(f"Gradient Flow: {activation}")
    ax.legend()
    ax.grid(True)

plt.tight_layout()
plt.show()

# Print a summary of gradients for each configuration
rows = []

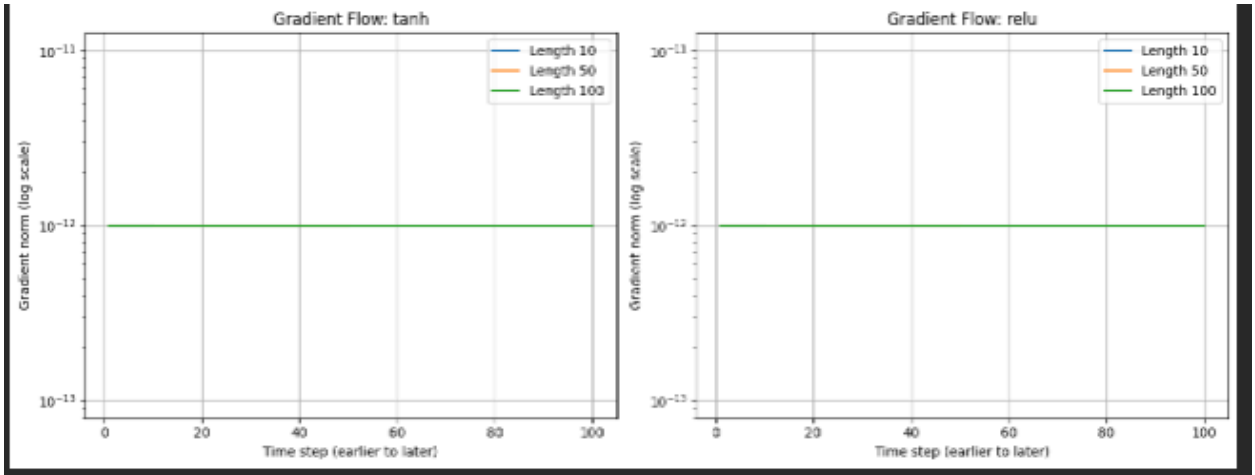
for (activation, length), norms in gradient_results.items():
    rows.append({
        "Activation": activation,
        "Sequence length": length,
        "Earliest gradient norm": norms[0],
        "Latest gradient norm": norms[-1],
        "Minimum gradient norm": norms.min(),
        "Maximum gradient norm": norms.max()
    })

summary = pd.DataFrame(rows)
print("\nGradient summary:")
print(summary.to_string(index=False, float_format=lambda v: f"{v:.6e}"))
```

Output:




```
Activation=tanh, length=10, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
Activation=tanh, length=50, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
Activation=tanh, length=100, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
Activation=relu, length=10, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
Activation=relu, length=50, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
Activation=relu, length=100, initial-state gradient norm=0.000000e+00, final-state gradient norm=0.000000e+00
```




```
Gradient summary:
Activation  Sequence length  Earliest gradient norm  Latest gradient norm  Minimum gradient norm  Maximum gradient norm
      tanh               10            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
      tanh               50            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
      tanh              100            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
      relu               10            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
      relu               50            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
      relu              100            0.000000e+00          0.000000e+00           0.000000e+00           0.000000e+00
```